

*Когда-нибудь все находят равновесие между  
страстями и равнодушием.*

Лосяш

# Алгоритмы и структуры данных

2025–2026 учебный год

## SET 5. Домашняя работа

### Хеширование и вероятностные структуры данных

| январь-февраль |    |    |    |    |    |    |
|----------------|----|----|----|----|----|----|
| 26             | 27 | 28 | 29 | 30 | 31 | 1  |
| 2              | 3  | 4  | 5  | 6  | 7  | 8  |
| 9              | 10 | 11 | 12 | 13 | 14 | 15 |
| 16             | 17 | 18 | 19 | 20 | 21 | 22 |
| 23             | 24 | 25 | 26 | 27 | 28 |    |
| пн             | вт | ср | чт | пт | сб | вс |

## Немного инструкций

Задания в рамках домашней работы SET 5 подразделяются на два блока:

1. *Блок А* «Аналитические задания» — задачи, связанные с разработкой, исследованием и применением различных способов хеширования.

Решения заданий Блока А оформляются в письменном виде в любом допустимом формате и загружаются для оценки в соответствующую форму раздела SET 5 на странице курса в LMS по ссылке: <https://edu.hse.ru/course/section.php?id=1378454>.

2. *Блок Р* «Задания на разработку» — задачи, связанные с реализацией алгоритмов и структур данных, основанных на использовании хеширования.

Решения заданий Блока Р загружаются в систему CODEFORCES и проходят автоматизированное тестирование. Для загрузки нужно перейти на <https://dsahse25.contest.codeforces.com> и выбрать соответствующее соревнование. Доступ к соревнованию предоставлен по тем же учетным данным, что и к системе Яндекс.Контест.

Домашняя работа SET 5 содержит 3 обязательных задачи в Блоке А и 3 обязательные задачи в Блоке Р. Баллы, которые можно получить за решение задач, распределены следующим образом:

| Блок А |     |        |    | Блок Р |    |    |
|--------|-----|--------|----|--------|----|----|
| A1     | A2b | A3     | A4 | P1     | P2 | P3 |
| 6      | 10  | 16(+7) | 6  | 15     | 12 | 8  |

Задачи, помеченные “b” не являются обязательными — баллы за их решение относятся к *бонусным*. Подтверждение решений бонусных задач сопровождается обязательной *устной защитой*.

## Важные даты

1. Домашняя работа SET 5 открыта с **19:30 26 января 2026 г.**
2. Прием решений завершается в **02:00 09 февраля 2026 г.**
3. Защита бонусной задачи A2b завершается в **21 февраля 2026 г.**

## Содержание

|            |                                    |    |
|------------|------------------------------------|----|
| Задача A1  | Анализ линейного пробирования      | 1  |
| Задача A2b | Взломщик!                          | 2  |
| Задача A3  | HyperMegaLogLog Pro Max++          | 3  |
| Задача A4  | Совместимость фильтров Блума       | 5  |
| Задача P5  | Хеш-таблица с закрытой адресацией  | 6  |
| Задача P6  | Реализация фильтра Блума для Ёжика | 9  |
| Задача P7  | Анаграммы                          | 12 |

Успехов!

## Задача А1. Анализ линейного пробирования

В хеш-таблице с открытой адресацией разрешение коллизий производится с помощью *линейного пробирования*. При удалении объекта из хеш-таблицы свободная ячейка получает значение **ERASED**, отличное от **NULL**, которое обозначает *пустое* значение.

Ниже приведены алгоритмы вставки, удаления и поиска, где  $M$  обозначает размер хеш-таблицы:

```
1  INSERT(key):
2      ind = hash(key) mod M
3
4      while (table[ind] != NULL)
5          if (table[ind] == key) return
6          ind = (ind + 1) mod M
7
8      table[ind] = key
```

```
1  DELETE(key):
2      ind = hash(key) mod M
3
4      while (table[ind] != NULL)
5          if (table[ind] == key)
6              table[ind] = ERASED
7              return
8          ind = (ind + 1) mod M
```

```
1  SEARCH(key):
2      ind = hash(key) mod M
3
4      while (table[ind] != NULL)
5          if (table[ind] == key)
6              return true
7          ind = (ind + 1) mod M
8      return false
```

### Система оценки

- 3 балла Приведенные выше алгоритмы вставки, удаления и поиска ключа имеют проблему, которая приводит к *долгому* выполнению некоторой(-ых) последовательности(-ей) этих операций.
  - Найдите такую(-ие) последовательность(-и) операций вставки, удаления и поиска.
  - Охарактеризуйте соответствующее состояние хеш-таблицы. Приведите примеры.
- 3 балла Предложите доработки (*кроме* перехеширования) исходных алгоритмов вставки, удаления и поиска, которые помогут исправить обнаруженную вами проблему.

## Задача A2b. Взломщик!

Для хеширования строковых ключей, которые могут содержать строчные/прописные латинские буквы и цифры, используется следующая полиномиальная хеш-функция, значение которой определяется числовым параметром  $p$ :

```
1  size_t hash(std::string key) {
2      const int p = ???;
3      long long h = 0, p_pow = 1;
4      for (size_t i = 0; i < s.length(); ++i) {
5          h += (s[i] - 'a' + 1) * p_pow;
6          p_pow *= p;
7      }
8      return h;
9  }
```

Для того, чтобы взломать хеш-функцию, требуется найти такой набор строк, который вызывает *коллизии* — одинаковые значения хеш-функции.

### Система оценки

1. 7 баллов Одним из способов подбора строк, вызывающих коллизии, является поиск так называемых нейтральных элементов — строк, значение хеш-функции которых *обращается в 0*. Разработайте и обоснуйте алгоритм поиска строк, состоящих из двух символов, которые будут являться нейтральными элементами. Представьте обоснование и реализацию алгоритма. *Ограничений на используемые языки программирования в этом задании нет.*
2. 3 балла Найдите нейтральные элементы для всех значений параметра  $p \leq 31$ .

## Задача А3. HyperMegaLogLog Pro Max++

Как известно, потоковые алгоритмы зачастую являются *аппроксимационными*. Выбирая такой алгоритм, мы жертвуем точностью ответа, но в обмен получаем преимущества по времени выполнения или занимаемой памяти. Такая аппроксимация представляет интерес, если мы можем зафиксировать некоторые пределы для ошибки этого алгоритма.

Вам предлагается исследовать один из таких алгоритмов **HyperLogLog**, который находит оценку  $N_t$  для частотного момента  $F_0^t$  — количества уникальных объектов в потоке на момент времени  $t$ . Чтобы провести успешное исследование, необходимо выполнить несколько важных этапов.

Разработка алгоритмов выполняется на языке C++. Ограничений на инструменты для визуализации и анализа результатов нет.

### Этап 1. Создание инфраструктуры

1. Разработайте класс **RandomStreamGen** для генерации потока данных  $S$ :
  - класс должен генерировать поток строк длиной до 30 символов;
  - допустимые символы: прописные и строчные латинские буквы, цифры 0–9, тире;
  - реализовать возможность разбиения потока на части (например, с шагом 5%, 10%, ...) для моделирования момента времени  $t$ .
2. Разработайте класс **HashFuncGen** для генерации хеш-функции  $h : U \rightarrow M = 2^{32}$ , где  $U$  — это множество строк, из которых формируется поток данных  $S$ :
  - хеш-функция должна давать приблизительно равномерное распределение значений;
  - рекомендуется использовать подходы из лекций 14 и 16 (линейные/полиномиальные хеши) или адаптировать существующие: например, MurmurHash (в некоторых версиях лежит в основе `std::hash`, FNV или SHA).

Дополнительно можно протестировать хеш-функцию на равномерность распределения.

### Этап 2. Реализация и оценка точности стандартного алгоритма HyperLogLog

Реализуйте любым удобным способом вероятностный алгоритм **HyperLogLog** для вычисления оценки  $N_t$ , а также функцию, вычисляющую точное число  $F_0^t$  уникальных объектов в потоке. Для алгоритма **HyperLogLog** требуется самостоятельно выбрать размер индекса субпотока — первые  $B$  бит (регистров). Кратко обоснуйте свой выбор  $B$  (для этого может понадобиться проведение нескольких тестовых запусков с разными значениями  $B$  для оценки того, как распределяются оставшиеся  $L - B$  бит по субпотокам). Сгенерируйте несколько потоков (количество и размеры потоков выберите самостоятельно) и:

1. Для каждой выбранной вами части каждого потока вычислите точное число  $F_0^t$  уникальных объектов и оценку  $N_t$ .
2. Определите выборочные статистики для всех сгенерированных потоков: среднее значение оценки  $\mathbb{E}(N_t)$ , а также стандартное отклонение  $\sigma_{N_t}$ .

Постройте следующие графики для визуализации результатов анализа точности **HyperLogLog**:

1. График №1 сравнения оценки  $N_t$  и  $F_0^t$ :
  - по оси  $X$ : номер шага (момент времени  $t$ ) или размер обработанной части потока;
  - по оси  $Y$ : количество уникальных элементов.

Соответственно, на этот график нанесите две линии: истинное значение  $F_0^t$  и оценка  $N_t$ .

2. График №2 статистик оценки:

- линия  $\mathbb{E}(N_t)$  и
- область неопределенности, которую можно представить в виде закрашенной области  $\mathbb{E}(N_t) + \sigma_{N_t}$  и  $\mathbb{E}(N_t) - \sigma_{N_t}$

### Этап 3. Анализ результатов работы стандартного алгоритма HyperLogLog

На этом этапе выполняется сравнение практических результатов с теоретическими оценками. Прокомментируйте:

1. Точность разработанного алгоритма. Оцените, насколько она укладывается в рамки теоретических отклонений (проверьте для  $\frac{1.04}{\sqrt{2^B}}$  и  $\frac{1.3}{\sqrt{2^B}}$ ).
2. Стабильность оценки (дисперсию), получаемой алгоритмом.
3. Эффективность выбранных вами констант и их влияние на результаты работы алгоритма.

### Этап 4 (опциональный). Усовершенствования HyperLogLog

Теперь когда вы проверили эффективность оригинального алгоритма, время внести в него что-то новое для достижения лучших результатов оценки  $N_t$ . Предложите модификацию стандартного алгоритма, например, для:

- повышения точности,
- уменьшения дисперсии или
- снижения потребления памяти (например, замена `std::vector` на `std::bitset`).

Кратко опишите суть предлагаемых изменений. Если используется стороннее улучшение, приведите ссылку на источник.

Повторите Этапы 2 и 3 для улучшенной версии алгоритма. При оптимизации памяти также проанализируйте потребление памяти стандартным алгоритмом.

Приложите ссылку на публичный репозиторий с:

- исходными кодами реализованных алгоритмов;
- исходными данными, которые использовались при тестировании и анализе;
- отчетом по оценке и анализу точности реализованных алгоритмов.

## **Система оценки**

1. 5 баллов Реализация инфраструктуры — классы `RandomStreamGen` и `HashFuncGen`.
2. 3 балла Реализация стандартного алгоритма `HyperLogLog` для вычисления оценки  $N_t$  и функции для вычисления точного числа  $F_0^t$  уникальных объектов в потоке.
3. 4 баллов Представление эмпирических данных анализа точности (график №1 и график №2).
4. 4 баллов Сравнительный анализ полученных эмпирических данных.
5. 7 баллов Исследование потенциала улучшения алгоритма `HyperLogLog` и последующее тестирование (Этап 4).

## **Замечание**

Эта задача является аналитической. В ее рамках вам нужно будет провести ряд экспериментов и сравнить их с известными теоретическими знаниями. Хотя от вас не ожидается подробный уровень эмпирических выводов, пожалуйста, уделите внимание тому, что ваши эксперименты и их результаты обоснованы.

## Задача А4. Совместимость фильтров Блума

Предположим, что по двум множествам объектов  $A$  и  $B$  создано два фильтра Блума одинакового размера —  $F(A)$  и  $F(B)$  соответственно. Путем выполнения поэлементного побитового И над фильтрами  $F(A)$  и  $F(B)$  был получен третий фильтр, который обозначен  $F(AB)$ .

### Система оценки

Ответьте на следующие вопросы:

1. 3 балла Верно ли, что  $F(AB)$  будет выдавать положительные ответы о принадлежности объектов из множества  $A \cap B$ ? Почему (нет)?
2. 3 балла Верно ли, что  $F(AB)$  будет в точности соответствовать другому фильтру, который будет получен в результате последовательной вставки объектов из множества  $A \cap B$ ? Почему (нет)? *Хеш-функции не меняются.*

## Задача Р1. Хеш-таблица с закрытой адресацией

|                         |                   |
|-------------------------|-------------------|
| Имя входного файла:     | стандартный ввод  |
| Имя выходного файла:    | стандартный вывод |
| Ограничение по времени: | 1 секунда         |
| Ограничение по памяти:  | 256 мегабайт      |

В ангаре у Пина царил хаос, как выразился бы Лосяш, «феноменальная энтропия». Великий механик Ромашковой долины готовился к починке Железной Няни и столкнулся с проблемой вселенского масштаба: в его гигантском ящике для деталей закончились свободные отсеки.

- О, майн Готт! - восклицал Пин, держа в крыльях редкую левостороннюю гайку.

- Куда мне положить эту деталь? Ящик номер семь занят правой резьбой, а ящик восемь - винтами! Компрессия невозможна!



На помощь пришел Лосяш, который как раз проходил мимо с томиком о дискретной математике. Он заметил, что Пин использует простейшую функцию распределения деталей и предложил элегантное решение проблемы коллизий.

- Друг мой технический, - поправил очки Лосяш. - Зачем же искать новый ящик, если можно расширить пространство внутри существующего? Если в седьмой отсек, предназначенный для гаек, поступает еще одна гайка, мы не будем выбрасывать предыдущую. Мы просто приварим... то есть, привяжем новую деталь к старой! Создадим, так сказать, структурную цепочку взаимосвязанных элементов.

Пин оценил идею, но для реализации системы учета ему нужен надежный алгоритм, с созданием которого вы должны помочь.

Хеширование объектов-деталей выполняется по полю Ключ, то есть при вставке объекта в хеш-таблицу вычисляется значение используемой хеш-функции от поля Ключ. Разрешение коллизий должно выполняться с помощью метода цепочек.

Создаваемый шаблонный класс `HashTable` параметризуется:

1. Типом используемых ключей `KeyType`.
2. Типом значения `ValueType`.
3. Типом используемого хешера `Func`. По умолчанию используется функциональный объект стандартной библиотеки `std::hash<KeyType>`.

В случае если таблица заполнена более чем на некоторое отсечное значение (коэффициент заполненности), выполняется перехеширование: удвоение емкости таблицы и перенос текущих объектов в новую таблицу. Коэффициент заполненности рассчитывается как отношение количества записей в хеш-таблице к количеству всех ячеек хеш-таблицы.



Скажем пару слов про «хешер». Это некоторый тип, который по ключу типа `KeyType` умеет выдавать значение типа `size_t`, которое можно получить, используя функциональный вызов.

Например: // пусть `hasher` имеет тип `Func`

`KeyType key = ...; // какой-то ключ`

`size_t num = hasher(key);`

Благодаря использованию шаблонов, тип `Func` может быть чем угодно — функцией, лямбдой или же классом, для которого перегружен оператор вызова `()`. Это позволяет пользователю вашего класса выбирать наиболее предпочтительный для него вариант хеш-функции. Если же пользователя устраивает стандартный вариант, то используется стандартный тип `std::hash<KeyType>`.

Обратите внимание, что сам по себе хешер всего лишь дает вам возможность получить для любого объекта некоторое число (и предоставляется пользователем класса, поскольку вы заранее не знаете, с какими типами будет использоваться ваша таблица), а вот как его использовать для организации быстрой хеш-таблицы — уже ваша забота. Однако же если хешер, например, возвращает для всех ключей число 0, то ясно, что это проблема пользователя, а от вас мало что зависит (тем не менее, ваш класс должен по-прежнему корректно работать в таких ситуациях, это должно отражаться только на времени работы). Поэтому вы можете считать, что хешер распределяет ключи по диапазону `size_t` достаточно равномерно (в предположении, что ключи случайны) — в частности, это верно для варианта по умолчанию: `std::hash<KeyType>`.

Ваш класс должен содержать следующие конструкторы и методы:

1. Конструктор по умолчанию. Размер таблицы по умолчанию принимается равным **100**. Коэффициент наполненности по умолчанию принимается равным **0.5**.
2. Конструктор с одним параметром — типом используемой хеш-функции.
3. Конструктор, который позволяет задавать максимальный размер таблицы и коэффициент наполненности, а также тип используемой хеш-функции. Последний параметр — тип используемой функции — может быть опущен. Коэффициент наполненности принимает значения от 0 до 1, где 1 включительно, а 0 нет. В случае передачи в конструктор неверного значения коэффициента наполненности, следует принять его равным дефолтному значению — 0.5.
4. Деструктор!
5. Константный метод `size`, возвращающий количество элементов в таблице.
6. Константный метод `capacity`, возвращающий текущее значение емкости таблицы.
7. Метод `insert`, который вставляет пару (ключ, значение) в хеш-таблицу. Принимает на вход два параметра (ключ, значение), вычисляет значение хеш-функции от поля Ключ (с поправкой на размер таблицы) и добавляет запись об этом объекте в таблицу. В случае коллизии, объект добавляется в конец соответствующей хешу цепочки. В случае, если объект с таким полем ключ уже имеется, то происходит обновление поля Значение. При превышении коэффициента наполненности ( $\text{size} / \text{capacity} > \text{coeff}$ ) происходит перехеширование. Тип возвращаемого значения `void`.
8. Метод `erase`, который принимает ключ и удаляет соответствующую пару (ключ, значение) из таблицы. В случае, если запись с таким ключом не найдена, то метод ничего не меняет.
9. Метод `find`, возвращающий указатель на значение, соответствующее переданному ключу хеш-таблицы, если ключ в ней присутствует, и `nullptr` в противном случае.
10. Оператор индексации (`operator []`), который возвращает ссылку на запись хеш-таблицы по значению переданного хеш-кода (односвязный список элементов) по ссылке. Если переданный хеш-код находится вне таблицы, то генерируется исключение `std::out_of_range`. Если переданный хеш-код указывает на пустую ячейку, то генерируется исключение `std::runtime_error`.

11. Метод `at`, который возвращает запись хеш-таблицы по значению переданного хеш-кода (одно-связный список элементов) по значению. Если переданный хеш-код находится вне таблицы, то генерируется исключение `std::out_of_range`. Если переданный хеш-код указывает на пустую ячейку, то генерируется исключение `std::runtime_error`.

Ваш класс должен состоять из узлов `Node` (тип элементов параметризуется — `KeyType`, `ValueType`) — односвязный список, содержащий объекты с одним хеш-кодом.

Структура `Node` должна содержать следующее:

1. Поле `key` — хранит ключ текущего узла (тип элемента — `KeyType`).
2. Поле `value` — хранит значение текущего узла (тип элемента — `ValueType`).
3. Поле `next` — хранит указатель на следующий узел с тем же хеш-кодом.
4. Конструктор, принимающий на вход два значения: `key` (тип — `KeyType`), `value` (тип — `ValueType`)

## Формат выходных данных

Вы должны прислать код, содержащий определение вашего класса.

Рекомендуемый публичный интерфейс (заголовочный файл) доступен в материалах соревнования. Подключать этот заголовочный файл в отправляемом решении **не нужно**.

Можно добавлять свои приватные поля или методы в класс/структуру.

## Система оценки

Всего будет не более  $5 \cdot 10^6$  операций с таблицей.

| Функция/Test                | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|-----------------------------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| Default Constructor         | + |   | + | + | + | + | + | + | + | +  | +  | +  | +  | +  |
| Constructor with Function   |   |   |   |   |   |   |   |   |   |    | +  |    |    |    |
| Constructor with Parameters |   | + |   |   |   |   |   |   |   |    |    |    |    |    |
| Destructor                  | + | + | + | + | + | + | + | + | + | +  | +  | +  | +  | +  |
| Size                        | + | + | + | + |   | + |   |   |   |    |    | +  |    |    |
| Capacity                    | + | + |   | + |   | + |   |   |   |    |    |    |    |    |
| Insert                      | + | + | + | + | + | + | + | + | + | +  | +  | +  | +  | +  |
| Erase                       |   |   |   |   |   | + |   |   |   |    |    | +  |    |    |
| Find                        |   |   |   | + | + | + | + |   | + |    | +  | +  |    |    |
| Operator []                 |   |   |   |   |   |   | + | + |   |    |    |    | +  |    |
| At                          |   |   |   |   |   |   |   |   | + | +  |    |    |    | +  |

| Группа | Баллы | Необходимые группы | Дополнительная информация | Информация о проверке |
|--------|-------|--------------------|---------------------------|-----------------------|
| 1      | 2     | —                  | —                         | первая ошибка         |
| 2      | 7     | 1                  | —                         | первая ошибка         |
| 3      | 3     | 1–2                | —                         | первая ошибка         |
| 4      | 3     | 1–3                | доп. проверки             | первая ошибка         |

## Задача Р2. Реализация фильтра Блума для Ёжика

|                         |                   |
|-------------------------|-------------------|
| Имя входного файла:     | стандартный ввод  |
| Имя выходного файла:    | стандартный вывод |
| Ограничение по времени: | 1 секунда         |
| Ограничение по памяти:  | 256 мегабайт      |

После роковой грозы Ёжик смог восстановить своё собрание фантиков. Теперь оно настолько огромное, что занимает целый шкаф с каталогами. Проверка наличия одного фантика занимает полдня перелистывания альбомов.

Крош, желая помочь другу, постоянно тащит новые мятые обертки:

- Ёжик, смотри! Раритет! Берем?

Ёжик в ужасе: если взять дубликат - нарушится гармония. Если полезть проверять в шкаф – это займёт ужасно много времени. Реализуйте вероятностный фильтр, который работает со строковыми объектами и поможет Ёжику сэкономить время на проверку новизны фантика.



Фильтр должен поддерживать:

1. Добавление строки в множество.
2. Тест принадлежности строки к множеству объектов.

Помимо этого, вам требуется добавить поддержку расчета доли ложно-положительных срабатываний, то есть значения, которое показывает отношение количества тестов принадлежности, которые дали ложно-положительный результат, к общему числу поступивших тестов принадлежности.

Реализуемый вами класс `BloomFilter` должен содержать следующие методы и конструкторы:

1. Конструктор, параметризуемый числом хеш-функций фильтра и числом ячеек фильтра.
2. Деструктор.
3. Метод `add`, который вставляет информацию о строке, с использованием хэш-функций, в множество. Принимает на вход значение строкового типа.
4. Метод `verify`, который проверяет, существует ли строка в Фильтре Блума.

Принимает на вход значение типа `std::string`, возвращает значение типа `bool`.

В случае, если Фильтр Блума показывает, что строка в нём находится, но при этом она не добавлялась — необходимо инкрементировать счетчик ложно-положительных значений на единицу.

- Метод `getFPRate`, который возвращает отношение количества ложно-положительных срабатываний к сумме всех запросов к функции `verify`.  
Тип возвращаемого значения — `double`.
- Константный метод `numberOfHashFunctions`, который возвращает количество хеш-функций для данного фильтра.
- Константный метод `numberOfBits`, который возвращает количество ячеек в данном фильтре.

Поскольку фильтр содержит информацию о количестве используемых хеш-функций, применяется следующий вариант получения  $k$  хеш-функций:

- используется объект стандартной библиотеки `std::hash`;
- при подсчете хеша для  $n$ -го хешера, где  $n > 0$ , добавляется некоторое подобие «соли»;
- в качестве «соли» выступает номер хеш-функции, приведенный к строке и добавленный в конец исходной строки.

Таким образом, например, вычисление 4 хеш-функций от строки «abcd» будет выглядеть следующим образом:

- `hash_0("abcd") = std::hash<std::string>{}("abcd");`
- `hash_1("abcd") = std::hash<std::string>{}("abcd1");`
- `hash_2("abcd") = std::hash<std::string>{}("abcd2");`
- `hash_3("abcd") = std::hash<std::string>{}("abcd3");`

В рамках задачи фильтр рассматривается в виде одного битового вектора.

## Формат входных данных

Вам нужно загрузить определение класса `BloomFilter`.

Загрузка неполного определения/реализации класса не гарантирует полное прохождение тестов в рамках группы.

## Формат выходных данных

Это задача-грейдер.

Вашей целью будет загрузить реализацию функций класса `BloomFilter` шаблоны всех функций даны. **Не требуется загружать что-то иное, кроме реализации предоставленных функций класса.**

Например, вы не можете добавить свои поля, как-то модифицировать файл `bloom_filter.h`. Тестироваться код будет с исходной версией этого файла, которая вам предоставлена.

## Система оценки

| Функция/Test          | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-----------------------|---|---|---|---|---|---|---|---|---|----|
| default constructor   | + | + | + | + | + | + | + | + | + | +  |
| destructor            | + | + | + | + | + |   | + | + | + | +  |
| add                   |   | + | + | + |   | + | + | + | + | +  |
| verify                |   | + | + | + |   | + | + | + | + | +  |
| getFPRate             |   |   |   |   | + | + | + | + | + | +  |
| numberOfHashFunctions | + |   |   |   |   |   |   |   |   |    |
| numberOfBits          | + |   |   |   |   |   |   |   |   |    |

| Группа | Баллы | Тесты | Дополнительная информация | Необходимые группы | Информация о проверке |
|--------|-------|-------|---------------------------|--------------------|-----------------------|
| 1      | 2     | 1—2   | —                         | —                  | первая ошибка         |
| 2      | 6     | 3—6   | —                         | 1                  | первая ошибка         |
| 3      | 2     | 7—10  | —                         | 1–2                | первая ошибка         |
| 4      | 2     | 11—12 | дополнительные проверки   | 1–3                | первая ошибка         |

## Замечание

- Используйте предложенный шаблон кода, который доступен в материалах соревнования. Там же приложен заголовочный файл.
- Не разрешается добавлять свои приватные поля и функции в реализацию класса, это приведёт к ошибке компиляции.

## Задача Р3. Анаграммы

|                         |                   |
|-------------------------|-------------------|
| Имя входного файла:     | стандартный ввод  |
| Имя выходного файла:    | стандартный вывод |
| Ограничение по времени: | 1 секунда         |
| Ограничение по памяти:  | 256 мегабайт      |

В Ромашковой долине - техническая революция! Пин и Лосяш наконец-то протянули телеграфную линию между домиком Совуньи и маяком, чтобы вовремя оповещать всех о готовности пирогов. Однако при первом же тесте выяснилось страшное: ветер запутал провода. Сигнал доходит, но порядок символов внутри сообщений безнадежно перемешивается. Совунья отправляет ингредиент: «СОЛЬ», а Пин получает «ЛЮСЬ» и в панике ищет рогатого друга.



Лосяш, изучив проблему, понял: хотя порядок букв нарушен, состав символов на определенных участках сигнала сохраняется. Чтобы восстановить связь, нужно найти самый длинный кусок сообщения, который, несмотря на перемешивание, совпадает по составу символов с эталонным сигналом настройки. Вам нужно найти максимальную длину такого участка сигнала, который является перестановкой букв из нашего проверочного кода.

По данным двум массивам (искаженный сигнал и эталон) требуется найти такое максимальное число  $k$ , что в первом и втором массивах существуют подотрезки длиной  $k$ , совпадающие как анаграммы. Под анаграммами в данном случае понимаются два слова, в которых можно как-то переставить буквы, чтобы они стали одинаковыми.

### Формат входных данных

В первой строке входного файла дано число  $n$  ( $1 \leq n \leq 5 \cdot 10^3$ ) – длина первого массива.

Во второй строке через пробел заданы  $n$  чисел  $a_i$  ( $1 \leq a_i \leq 10^6$ ) – первый массив.

В третьей строке входного файла дано число  $m$  ( $1 \leq m \leq 5 \cdot 10^3$ ) – длина второго массива.

В четвертой строке через пробел заданы  $m$  чисел  $b_i$  ( $1 \leq b_i \leq 10^6$ ) – второй массив.

### Формат выходных данных

В единственной строке выведите три неотрицательных числа  $k$ ,  $i$ ,  $j$  – максимальная длина подотрезков, совпадающих как анаграммы, а также начало отрезка в первом массиве и во втором соответственно. Если максимальная длина подотрезка равна 0, следующие два числа в выходном файле должны равняться -1.

### Система оценки

| Подзадача | Баллы | Дополнительные ограничения | Необходимые подзадачи | Информация о проверке |
|-----------|-------|----------------------------|-----------------------|-----------------------|
| 0         | 0     | тесты из условия           | —                     | полная                |
| 1         | 1     | $1 \leq n, m \leq 10$      | 0                     | первая ошибка         |
| 2         | 1     | $10 \leq n, m \leq 100$    | 1                     | первая ошибка         |
| 3         | 2     | $100 \leq n, m \leq 500$   | 1–2                   | первая ошибка         |
| 4         | 2     | $500 \leq n, m \leq 1000$  | 1–3                   | первая ошибка         |
| 5         | 2     | $1000 \leq n, m \leq 5000$ | 1–4                   | первая ошибка         |

## Примеры

| стандартный ввод                     | стандартный вывод |
|--------------------------------------|-------------------|
| 6<br>1 5 9 8 7 5<br>6<br>5 7 8 4 5 1 | 3 4 1             |
| 6<br>5 4 3 2 1 9<br>6<br>3 4 5 7 6 5 | 3 1 1             |